

# Bologna City Mapping As A Graph Network

Singh Damneet Gill

N. Matricola 207056

AI

**Abstract**—This paper presents the construction and analysis of a graph-based representation of the city of Bologna using Neo4j. Publicly available urban datasets are integrated and processed through a Python preprocessing pipeline in order to generate a spatial network that models streets, pedestrian paths, cycling routes, and points of interest. The resulting graph structure enables the application of graph analytics techniques to explore different mobility scenarios. Using Neo4j and the Graph Data Science library, several pathfinding and urban analysis queries are performed to simulate the needs of different categories of users, such as pedestrians, elderly citizens, and tourists. The results demonstrate how graph-based modeling can support urban analysis, route optimization, and accessibility studies in complex city environments.

## I. DATASETS

To implement this project, different datasets were used:

- [1] **carta-tecnica-comunale-percorsi-pedonali** available on the Open Data portal of the Municipality of Bologna, representing pedestrian paths in the city.
- [2] **piste-ciclopeditoni** representing cycling paths, available on the Open Data portal of the Municipality of Bologna.
- [4] **aree-stradali** representing road routes in Bologna, available on the Open Data portal of the Municipality of Bologna.
- [5] **portici** representing one of the most distinctive architectural features of the city, available on the Open Data portal of Bologna.
- [3] **Elenco parchi e giardini** representing green areas in the city, available on the Open Data portal of the Municipality of Bologna.
- **DTM 0,5x0,5m** representing altitude data, available on the Geoportal of the Emilia-Romagna Region.

## II. DATA PROCESSING

### A. Graph Preprocessing

After importing the datasets from the sources in GeoJSON format, they are reprojected to the EPSG:3857 coordinate reference system to ensure spatial consistency.

The geometries contained in the datasets are then normalized by extracting linear elements from different geometry types. Since some paths may be shared by multiple mobility types (pedestrian, cycling, or road), they are aligned and merged to avoid creating multiple edges representing the same street.

Once a unified network is created, each segment is classified according to the type of mobility it supports (pedestrian, cycling, or road traffic). This classification is performed by

comparing the overlap between the original datasets and the merged network.

Segments intersecting with the portico dataset are then marked through a specific flag.

From the merged network, a base graph is generated using the `segments_to_graph` method from the [6] **city2graph** library. In the resulting graph, nodes represent street intersections while edges represent street segments.

### B. Elevation

To enrich the spatial graph, elevation information is added. The dataset is composed of .tif raster images together with .tfw and .prj files containing georeferencing information.

As done with the other datasets, a coordinate transformation is applied to ensure compatibility.

For each node in the graph, the coordinates are converted into the raster coordinate reference system. The corresponding elevation value is then retrieved and stored as a node attribute.

### C. OSMWayNode Graph

To ensure a standardized structure, the graph is converted into an OSMWayNode graph.

Nodes from the base graph are transformed into OSMWayNode nodes, containing latitude, longitude, and altitude as attributes.

In the second step, Way nodes are created. These represent the edges of the base graph and store information such as:

street length slope percentage altitude difference

Each Way node is connected to the corresponding OSMWayNode nodes through the MEMBER\_OF relationship.

To represent additional metadata, Tag nodes are introduced.

For this project, four different tags are used:

- Pedonale
- Stradale
- Ciclabile
- Portico

### D. POI

Green areas are added as Points of Interest (POI).

First, parks are approximated by computing their centroids, which are used to create POI nodes.

Each POI is then connected to the nearest OSMWayNode through a NEAR relationship.

### III. GRAPH STRUCTURE

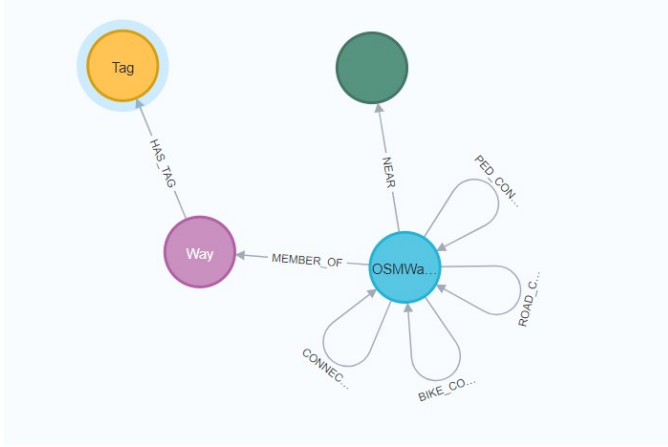


Fig. 1. Graph structure

To export the graph generated by the Python script into Neo4j, the graph was structured as follows:

- Constraints: For each type of node, a constraint enforcing the uniqueness of the node identifier was created.

```
CREATE CONSTRAINT osm_node_id_unique IF NOT EXISTS
FOR (n:OSMWayNode) REQUIRE n.id IS UNIQUE;

CREATE CONSTRAINT way_node_id_unique IF NOT EXISTS
FOR (w:Way) REQUIRE w.id IS UNIQUE;

CREATE CONSTRAINT poi_node_id_unique IF NOT EXISTS
FOR (p:POI) REQUIRE p.id IS UNIQUE;

CREATE CONSTRAINT tag_node_key_value_unique IF NOT
EXISTS
FOR (t:Tag) REQUIRE (t.key, t.value) IS UNIQUE
```

- Nodes: every node was uploaded with its relative properties.

|   | count(n) |
|---|----------|
| 1 | 62703    |

Fig. 2. Total Number Of Nodes

- **OSMWayNode**: 12092 nodes representing street intersections
- **Way**: 15256 nodes representing street segments

```
{
  "identity": 0,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "elev": 43.97637176513672,
    "x": 1250108.633918686,
    "y": 5548182.174330614,
    "id": "0"
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:0"
}
```

Fig. 3. OSM

```
{
  "identity": 24150,
  "labels": [
    "Way"
  ],
  "properties": {
    "delta_h": 0.4099884033203125,
    "modes": [
      "stradale"
    ],
    "slope_pct": 0.7106679797010852,
    "length": 57.690569299711264,
    "id": "way_0"
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:24150"
}
```

Fig. 4. Way

- **TAG**: 4 nodes representing the mobility types (pedestrian, cycling, road) and the portico attribute.

```
{
  "identity": 39868,
  "labels": [
    "Tag"
  ],
  "properties": {
    "value": "stradale",
    "key": "highway"
  },
  "elementId": "4:57d95bfa-d78a-41e7-8c4a-2e089cd9628c:39868"
}
```

Fig. 5. TAG

- **POI**: nodes representing points of interest in the city. In this project, only parks were included, but since each POI node contains a subtype property, the model can easily be extended in future developments.

```
{
  "identity": 39868,
  "labels": [
    "Tag"
  ],
  "properties": {
    "value": "stradale",
    "key": "highway"
  },
  "elementId": "4:57d95bfa-d78a-41e7-8c4a-2e089cd9628c:39868"
}
```

Fig. 6. Parks

- Edges: In addition to the edges generated by the Python script, several additional relationship types were created to represent different mobility layers. This allows the graph to be treated as a multilayer network, where each layer corresponds to a mobility type. The following relationships were defined:

- **CONNECTED\_TO**
- **PED\_CONNECTED\_TO**
- **BIKE\_CONNECTED\_TO**
- **ROAD\_CONNECTED\_TO**
- **MEMBER\_OF**
- **HAS\_TAG**
- **NEAR**

For simplicity, the properties length, slope\_pct, and delta\_h were also added to the mobility relationships.

#### IV. PATHFINDING ANALYSIS

Once the urban graph has been constructed, several pathfinding analyses can be performed using the tools provided by Neo4j and the Graph Data Science library. These analyses allow the simulation of different scenarios representing the needs of various types of users.

##### A. Shortest Path Between Two Nodes

The most basic analysis consists of finding the shortest path between two nodes of the network. This operation allows the identification of the minimal route between two intersections in the city.

```
MATCH (start:OSMWayNode {id: '15420'}), (end:OSMWayNode {id: '1973'})
MATCH p = shortestPath((start)-[:CONNECTED_TO*..100]-(end))
RETURN p
```

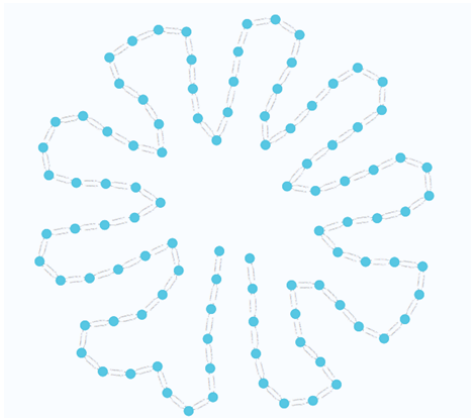


Fig. 7.

##### B. Closed Routes for Running

A possible application of the graph is the generation of closed routes of a predefined length. This scenario can represent a runner who wants to follow a circular path of approximately five kilometers.

```
MATCH (start:OSMWayNode {id:'1973'})
MATCH p = (start)-[:CONNECTED_TO*..30]->(start)
WITH p,
reduce(total = 0, r IN relationships(p) | total + r.length)
AS distance
WHERE distance > 4900 AND distance < 5100
RETURN distance, p
LIMIT 1
```



Fig. 8.

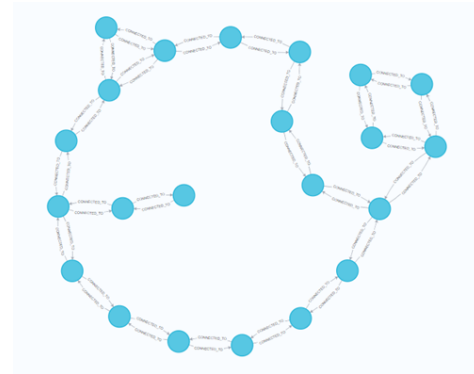


Fig. 9.

##### C. Pedestrian Walks of a Specific Length

The multilayer structure of the graph allows filtering the network based on mobility type. In this case, only pedestrian paths are considered in order to generate a walking route of a predefined distance.

```
MATCH (start:OSMWayNode {id:'4705'})
MATCH p = (start)-[:PED_CONNECTED_TO*..30]->()
WITH p,
reduce(total = 0, r IN relationships(p) | total + r.length)
AS distance
WHERE distance > 500 and distance < 600
RETURN distance, p
LIMIT 1
```

| distance          | p                                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------------|
| 531.4095459447623 | {         "start": {           "identity": 4705,           "labels": [             "OSMWayNode"           ], |

Fig. 10.

#### D. Accessible Routes for Elderly Users

Accessibility can be improved by considering the slope of each street segment. Using the Dijkstra algorithm provided by the Graph Data Science library, it is possible to compute the route that minimizes the overall slope.

```
CALL gds.graph.project(
  'slopeGraph',
  'OSMWayNode',
  {
    PED_CONNECTED_TO: {
      orientation: 'UNDIRECTED',
      properties: {
        slope_pct: { property: 'slope_pct'},
        length: { property: 'length' } } } } );

MATCH (start:OSMWayNode {id: '851'})

CALL gds.allShortestPaths.dijkstra.stream('slopeGraph', {
  sourceNode: start,
  relationshipWeightProperty: 'slope_pct'
})
YIELD targetNode, totalCost, nodeIds
WITH gds.util.asNode(targetNode) AS destination,
  totalCost AS penidenzaTotale,
  nodeIds
WHERE destination.id <> '851'

UNWIND range(0, size(nodeIds)-2) AS i
WITH destination, penidenzaTotale, nodeIds, nodeIds[i] AS id1
  , nodeIds[i+1] AS id2
MATCH (n1)-[r:PED_CONNECTED_TO]-(n2)
WHERE id(n1) = id1 AND id(n2) = id2
WITH destination,
  penidenzaTotale,
  sum(r.length) AS lunghezzaReale,
  [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS
  percorso_nodi

WHERE lunghezzaReale < 600 AND lunghezzaReale > 300

RETURN destination.id AS id_destinazione,
  lunghezzaReale AS metri,
  penidenzaTotale AS penidenza_accumulata,
  (penidenzaTotale / lunghezzaReale) * 100 AS
  penidenza_media_percentuale,
  percorso_nodi
ORDER BY penidenza_media_percentuale ASC
LIMIT 1;
```

| id_destinazione | metri             | penidenza_accumulata | penidenza_media_percentuale | percorso_nodi         |
|-----------------|-------------------|----------------------|-----------------------------|-----------------------|
| "106"           | 523.6105814844776 | 0.62513488942006     | 0.11928627761288077         | ["101", "102", "104"] |

Fig. 11.

This allows the computation of paths that are easier to walk for elderly users or people with mobility constraints.

#### E. Finding the Nearest Park

Another useful scenario consists of identifying the nearest green area while considering accessibility. In this case, parks are represented as Points of Interest connected to the street network through NEAR relationships. By combining these relationships with the pedestrian graph, the most accessible route to the closest park can be computed.

```
MATCH ()-[r:PED_CONNECTED_TO]->()
SET r.cost = r.length * (1 + abs(r.slope_pct)/10);

MATCH ()-[r:NEAR]->()
SET r.cost = 1

CALL {
```

```
MATCH (source:OSMWayNode)-[r:PED_CONNECTED_TO]-(target:
  OSMWayNode)
RETURN source, target, type(r) AS relType, r.cost AS cost
UNION
MATCH (source:OSMWayNode)-[r:NEAR]->(target:POI)
RETURN source, target, type(r) AS relType, r.cost AS cost
}
WITH gds.graph.project(
  'pedestrianGraph',
  source,
  target,
  {
    sourceNodeLabels: labels(source),
    targetNodeLabels: labels(target),
    relationshipType: relType,
    relationshipProperties: { cost: cost }
  }
) AS g
RETURN g.nodeCount AS nodeCount, g.relationshipCount AS
  relationshipCount

MATCH (start:OSMWayNode {id: '136'})
CALL gds.allShortestPaths.dijkstra.stream('pedestrianGraph',
  {
    sourceNode: start,
    relationshipWeightProperty: 'cost'
  })
YIELD targetNode, totalCost
WHERE 'POI' IN labels(gds.util.asNode(targetNode))
RETURN gds.util.asNode(targetNode).id AS poiName, totalCost
ORDER BY totalCost ASC
LIMIT 1
```

|   | poiName    | totalCost         |
|---|------------|-------------------|
| 1 | "park_788" | 140.5482889694528 |

Fig. 12.

#### F. Tourist Routes with Portico Coverage

Bologna is internationally known for its extensive network of porticos. In rainy conditions, it may be useful to prefer routes that are covered by porticos. This can be modeled by assigning a lower traversal cost to edges that intersect with portico segments and computing the shortest weighted path accordingly.

```
MATCH (n1:OSMWayNode)-[r:CONNECTED_TO]-(n2:OSMWayNode)
OPTIONAL MATCH (n1)-[:MEMBER_OF]->(w:Way)-[:HAS_TAG]->(t:Tag
  {value: 'portico'})
WHERE (n2)-[:MEMBER_OF]->(w)
SET r.is_covered = CASE WHEN t IS NOT NULL THEN 1 ELSE 0 END

r.weighted_cost = CASE WHEN t IS NOT NULL THEN r.length
  ELSE r.length * 3 END;

CALL gds.graph.project(
  'coveredGraph',
  'OSMWayNode',
  'CONNECTED_TO',
  {
    relationshipProperties: ['length', 'weighted_cost', '
      is_covered']
  })
MATCH (source:OSMWayNode {id: '1568'}),
  (target:OSMWayNode {id: '1960'})

CALL gds.shortestPath.dijkstra.stream('coveredGraph', {
  sourceNode: source,
  targetNode: target,
  relationshipWeightProperty: 'weighted_cost'
})
YIELD nodeIds, totalCost

WITH nodeIds, totalCost, [nodeId IN nodeIds | gds.util.
  asNode(nodeId).id] AS path_ids
```

```

UNWIND range(0, size(nodeIds)-2) AS i
MATCH (n1)-[r:CONNECTED_TO]-(n2)
WHERE id(n1) = nodeIds[i] AND id(n2) = nodeIds[i+1]

WITH path_ids, totalCost, sum(r.is_covered) AS
total_covered_arcs

RETURN path_ids,
totalCost AS costo_pesato,
total_covered_arcs

```

```

path_ids
["1997","1960"]

```

Fig. 13.

|                 | costo_pesato       | total_covered_arcs |
|-----------------|--------------------|--------------------|
| ["1997","1960"] | 11467.720626494565 | 16                 |

Fig. 14.

## V. URBAN ANALYSIS

In addition to pathfinding analysis, the constructed graph can also be used to study the structural properties of the urban network. Graph analytics techniques allow the identification of central areas of the city as well as the most important intersections within the mobility network.

### A. Community Detection

One of the most useful graph analysis techniques for urban networks is community detection, which identifies clusters of nodes that are more densely connected to each other than to the rest of the graph. In an urban context, these clusters often correspond to neighborhoods or districts, where streets are highly interconnected locally but less connected with distant areas. To perform this analysis, the Louvain community detection algorithm available in the Neo4j Graph Data Science library was applied to the urban graph. The Louvain algorithm assigns a community identifier to each node in the network. By aggregating nodes belonging to the same community and computing their average geographic coordinates, it becomes possible to approximate the spatial location of each detected district.

- Santa Viola / Barca
- San Donato / Pilastro
- Corticella / Navile Nord
- Centro – Bolognina
- Saffi / Ospedale Maggiore
- Borgo Panigale / Airport

This analysis shows how graph clustering can approximate urban zoning.

```

CALL gds.graph.project (
'urbanGraph',
'OSMWayNode',
'CONNECTED_TO',

```

```

{
relationshipProperties: ['length']
}
);

CALL gds.louvain.stream('urbanGraph')
YIELD nodeId, communityId
RETURN communityId, count(*)
ORDER BY count(*) DESC;
CALL gds.louvain.write('urbanGraph', { writeProperty: '
quartiere_id' });

MATCH (n:OSMWayNode)
WHERE n.quartiere_id IS NOT NULL
RETURN n.quartiere_id AS ID_Community,
avg(n.x) AS Latitudine_Media,
avg(n.y) AS Longitudine_Media,
count(*) AS Numero_Nodi
ORDER BY Numero_Nodi DESC

```

| ID_Community | Latitudine_Media | Longitudine_Media | Numero_Nodi |
|--------------|------------------|-------------------|-------------|
| 7329         | 120584.461719408 | 554306.20533336   | 600         |
| 18111        | 120221.051104717 | 553079.80811055   | 587         |
| 16121        | 120412.88803783  | 554279.249176219  | 567         |
| 10073        | 120270.852729487 | 554386.212419325  | 562         |
| 7649         | 120194.78380739  | 554323.71151114   | 512         |
| 2588         | 120736.05402217  | 554371.46829115   | 504         |

Fig. 15.



Fig. 16.

| Community | Latitudine | Longitudine | Zona / quartiere di             | Nodi |
|-----------|------------|-------------|---------------------------------|------|
| 7329      | 445.177    | 113.115     | zona Santa Viola / Barca        | 600  |
| 18111     | 444.979    | 113.621     | zona San Donato / Pilastro      | 587  |
| 16121     | 445.245    | 113.556     | zona Corticella / Navile nord   | 567  |
| 10073     | 445.064    | 113.364     | zona Centro - Bolognina         | 562  |
| 7649      | 445.128    | 113.245     | zona Saffi / Ospedale Maggiore  | 512  |
| 2588      | 445.120    | 112.910     | zona Borgo Panigale / aeroporto | 504  |

Fig. 17.

### B. Closeness Centrality: Identifying Highly Accessible Nodes

Closeness centrality measures how easily a node can reach all other nodes in the network. In the context of an urban

transportation graph, this metric identifies intersections that are globally well-positioned within the network, allowing efficient access to many parts of the city.

Formally, closeness centrality is defined as the inverse of the average shortest-path distance between a node and all other nodes in the graph. Nodes with high closeness values are located near the center of the network topology.

By applying the Closeness Centrality algorithm to the Bologna urban graph, the analysis reveals the nodes that provide the shortest average travel distance to the rest of the network. To further refine the analysis, a secondary graph representing only road-based connectivity was created using ROAD\_CONNECTED\_TO relationships. This allows the identification of central nodes specifically within the vehicular road network.

```
CALL gds.closeness.stream('urbanGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId), score
ORDER BY score DESC
LIMIT 1

CALL gds.graph.project(
  'roadGraph',
  'OSMWayNode',
  'ROAD_CONNECTED_TO',
  {
    relationshipProperties: ['length']
  }
);

CALL gds.closeness.stream('roadGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId), score
ORDER BY score DESC
LIMIT 1
```

```
{
  "identity": 526,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "x": 1252829.9282097921,
    "y": 5543680.801617972,
    "id": "526",
    "elev": 56.36629867553711
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:526"
}
```

Fig. 18.

```
{
  "identity": 612,
  "labels": [
    "OSMWayNode"
  ],
  "properties": {
    "x": 1253490.8293398009,
    "y": 5549739.389205114,
    "id": "612",
    "quartiere_id": 1409,
    "elev": 37.012298583964375
  },
  "elementId": "4:dc32b2ce-c39e-47d4-86f5-1c2d1d8df6b8:612"
}
```

Fig. 19.

### C. Betweenness Centrality: Detecting Strategic Intersections

While closeness centrality identifies globally accessible nodes, betweenness centrality focuses on nodes that play a crucial role as intermediaries in shortest paths.



Fig. 20.

A node with high betweenness centrality appears frequently in the shortest paths between other nodes. In transportation networks, such nodes represent critical intersections that act as bridges between different parts of the city.

```
CALL gds.betweenness.stream('urbanGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS incrocio, score
ORDER BY score DESC
LIMIT 10

CALL gds.betweenness.stream('roadGraph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).id AS incrocio, score
ORDER BY score DESC
LIMIT 10
```

|   | incrocio | score              |
|---|----------|--------------------|
| 1 | "4363"   | 57342777.365801826 |
| 2 | "5277"   | 55494143.31022505  |
| 3 | "5121"   | 51410797.38048203  |
| 4 | "5147"   | 51310793.400054306 |
| 5 | "5113"   | 50919510.32275317  |
| 6 | "5112"   | 50905526.819343366 |

Fig. 21.

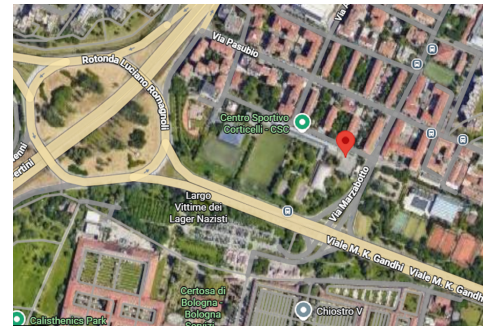


Fig. 22.

## VI. CONCLUSION

This work demonstrates how open urban datasets can be integrated and transformed into a graph representation of a

|   | incrocio | score              |
|---|----------|--------------------|
| 1 | "10568"  | 53145243.29468349  |
| 2 | "5583"   | 50407345.286845475 |
| 3 | "5058"   | 48517130.598985896 |
| 4 | "11650"  | 48362878.35089885  |
| 5 | "11769"  | 48207733.158191614 |
| 6 | "4789"   | 47869602.78562825  |

Fig. 23.

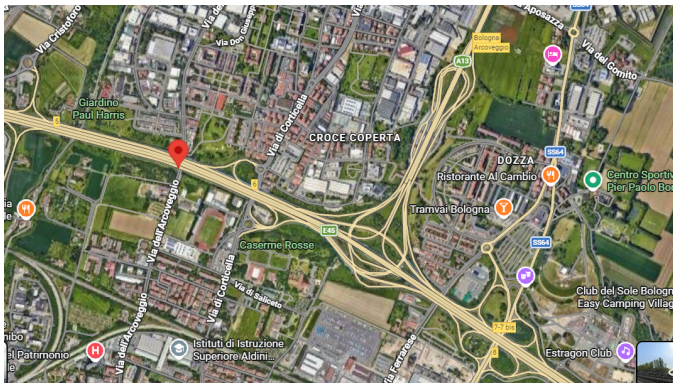


Fig. 24.

city using Neo4j. By combining spatial preprocessing techniques with graph modeling, it is possible to build a detailed representation of the urban mobility network of Bologna.

The resulting graph structure enables the analysis of multiple mobility scenarios. Through the use of pathfinding algorithms and graph analytics techniques, different types of users can be simulated, including pedestrians, runners, elderly citizens, and tourists. The multilayer structure of the network allows the separation of pedestrian, cycling, and road infrastructures while maintaining their relationships within a unified graph model.

Furthermore, the integration of elevation data and portico coverage demonstrates how additional urban features can be incorporated to support accessibility and route optimization. These enriched attributes allow the computation of routes that consider not only distance but also slope and environmental conditions.

Overall, this project highlights the potential of graph databases and graph analytics tools for urban analysis and smart mobility applications. Future developments could extend the model by including additional points of interest, real-time mobility data, or traffic information, enabling more advanced urban planning and decision-support systems.

## REFERENCES

- [1] title = Percorsi pedonali, url =<https://opendata.comune.bologna.it/explore/dataset/carta-tecnica-comunale-percorsi-pedonali/information/?disjunctive.descrizione&disjunctive.origine>
- [2] title = Piste e itinerari ciclabili, url =<https://opendata.comune.bologna.it/explore/dataset/piste-ciclopeditonali/export/?disjunctive.duso&disjunctive.anno>
- [3] title = Elenco parchi e giardini, url =<https://opendata.comune.bologna.it/explore/dataset/carta-tecnica-comunale-toponimi-parchi-e-giardini/export/>
- [4] title = Aree stradali, url =<https://opendata.comune.bologna.it/explore/dataset/aree-stradali/export/?disjunctive.descrizione&disjunctive.origine>
- [5] title = Portici di Bologna, url =<https://opendata.comune.bologna.it/explore/dataset/portici-di-bologna/export/?disjunctive.descrizione&disjunctive.origine>
- [6] = @softwaresato2025city2graph, title = City2Graph: Transform geospatial relations into graphs for spatial network analysis and Graph Neural Networks, author = Sato, Yuta, year = 2025, url = <https://github.com/c2g-dev/city2graph>, doi = 10.5281/zenodo.15858845,